

Employees API Kubernetes Deployment Documentation

1. Requirement Understanding

The goal of this assignment was to design, containerize, and deploy an Employee Management API on Kubernetes using industry-standard DevOps practices.

The solution was expected to:

- Host a Node.js (Express) application serving employee information.
- Use MySQL as the persistent backend database.
- Package the application using Docker.
- Deploy resources through Kubernetes and Helm.
- Enable automatic scaling based on resource utilization.
- Ensure application availability during updates.
- Secure configuration and credentials management.
- Provide external access to application endpoints.

2. Assumptions

The implementation was completed under the following assumptions:

- A functional Kubernetes cluster is available.
- Helm 3.x is installed and configured.
- Docker Hub is used for image storage and distribution.
- Metrics Server is installed to support HPA.
- MySQL data must persist across pod restarts.
- Application traffic is routed through an NGINX Ingress Controller.
- CPU utilization is used as the scaling metric.
- One MySQL instance is sufficient for the current workload.

3. Solution Overview

The solution adopts a cloud-native architecture consisting of Docker, Kubernetes, Helm, and MySQL.

Application Layer

The Employee API is developed using Node.js and Express.js.

Responsibilities:

- Handle incoming HTTP requests.
- Query employee information from MySQL.
- Return structured JSON responses.
- Expose a health-check endpoint for Kubernetes monitoring.

Supported endpoints:

- GET /employees
- GET /health

Database Layer

MySQL is deployed as a StatefulSet to provide persistent storage and stable network identity.

The database initialization process is automated using the provided schema file which:

- Creates required tables.
- Inserts sample employee records.
- Prepares the environment for application startup.

Containerization Strategy

The application is packaged into a lightweight Docker image based on Node.js Alpine.

Key benefits include:

- Consistent execution environment.
- Simplified deployment process.
- Reduced image size.
- Improved portability across environments.

Kubernetes Components

Deployment

The API workload is deployed using a Kubernetes Deployment.

Configuration:

- Initial Replicas: 4
- Rolling Update Strategy Enabled
- CPU and Memory Requests/Limits Defined

Services

Two services are used:

Employee API Service (ClusterIP)

- Internal service discovery
- Pod load balancing

MySQL Service

- Connectivity to StatefulSet database pods

ConfigMaps

Used to store non-sensitive runtime configuration:

- Database Host
- Database Port
- Database Name

Secrets

Used to securely store:

- Database Username
- Database Password

Ingress

NGINX Ingress provides external access to the application.

Example endpoint:

`http://35.231.79.130/employees`

Auto Scaling

Horizontal Pod Autoscaler (HPA) is configured with:

- Minimum Replicas: 4
- Maximum Replicas: 10
- CPU Threshold: 70%

This enables automatic scale-out during peak traffic and scale-in during low utilization periods.

Deployment Strategy

Rolling updates are configured to avoid service interruption.

Configuration:

maxSurge: 1
maxUnavailable: 1

Benefits:

- Continuous service availability during deployments.
- Controlled pod replacement.
- Safer production upgrades.

4. Justification for Resource Utilization

Node.js & Express

Selected because of:

- Lightweight runtime.
- Fast API development.
- Strong community support.

MySQL

Chosen due to:

- Relational data consistency.
- Proven reliability.
- Simple integration with Node.js applications.

Docker

Used to:

- Standardize deployments.
- Eliminate environment inconsistencies.
- Improve portability.

Kubernetes

Provides:

- High availability.
- Self-healing capabilities.
- Automated scaling.
- Declarative infrastructure management.

Helm

Helm was used because it:

- Simplifies Kubernetes deployments.
- Supports reusable templates.
- Enables versioned releases and rollbacks.

StatefulSet

Chosen for MySQL because databases require:

- Persistent storage.
- Stable pod identity.
- Predictable networking.

Persistent Storage

A 10Gi Persistent Volume was allocated to:

- Preserve database records.
- Prevent data loss.
- Support pod recreation scenarios.

ConfigMaps and Secrets

These resources improve maintainability by separating configuration from application code while ensuring sensitive information remains protected.

Horizontal Pod Autoscaler

HPA helps maintain application responsiveness while optimizing infrastructure utilization through automatic scaling.

NGINX Ingress

NGINX Ingress was selected to provide:

- Centralized traffic management.
- External application exposure.
- Future HTTPS/TLS support.

Conclusion

The final solution delivers a scalable, containerized Employee Management API running on Kubernetes.

By combining Docker, Helm, StatefulSets, Persistent Volumes, ConfigMaps, Secrets, Ingress, and Horizontal Pod Autoscaling, the deployment satisfies the assignment requirements while following modern DevOps and cloud-native architecture principles.